

AI Use Log

This document tracks each version of the project, including prompts used, AI model used, observations, feedback, and next steps.

Table of Contents

[Version 1](#) (5/11/26)

[Version 2](#)

Version 1

Prompt Used

Create a website where a user can upload photo, click on the uploaded photo to add text, and the user can download the the edited image.

- text that the user adds must be shiftable based on drag-and-drop and the text should be automatically resizable based of drag-and-drop of the corners of the text box
- the default text should be the Impact font with white text and black borders
- font can be modified with a toggle bar above the text box that has a dropdown menu of the following other fonts: Arial, Comic Sans, Helvetica, Montserrat
- allow users to choose between no border and border (the default option should be borders)

AI Model Used

Tim - I used Claude Opus 4.6 in high effort plan mode.

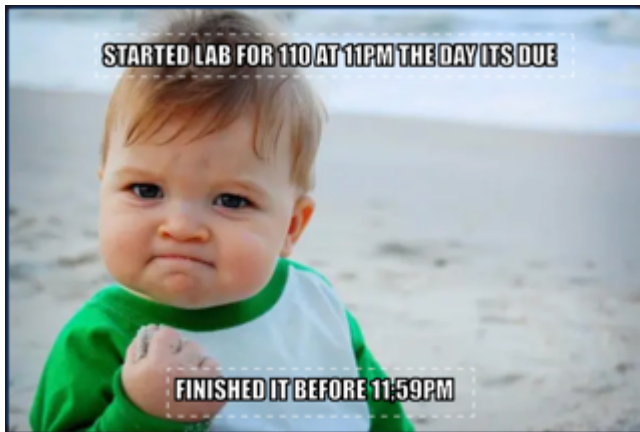
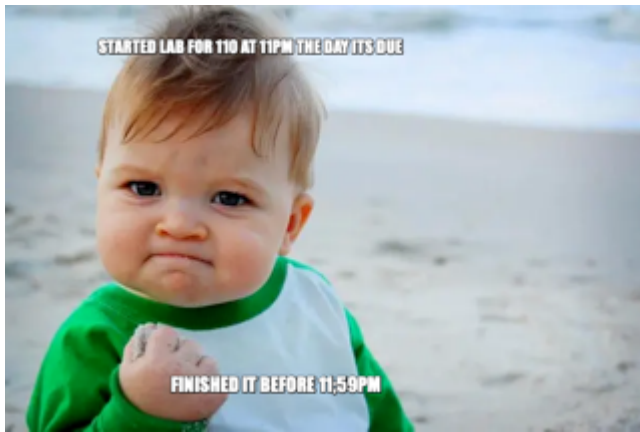
Observations

Goals Achieved

- Text added by the user is draggable and repositionable via drag-and-drop
- Text box corners support drag-and-drop resizing to automatically resize the text box
- Default font is Impact with white text and black borders
- A toggle bar above the text box includes a dropdown menu allowing font selection from: Arial, Comic Sans, Helvetica, and Montserrat
- Users can toggle between no border and border styles, with border enabled as the default
- Download works on Mac

Goals Not Achieved (Evaluate and re-implement)

- Font size does not automatically resize based on the text box size — font resizability was not achieved.
- Downloaded meme has a different text appearance than what is seen on the website.



Next steps for v2

- Fix font automatic resizing relative to text-box size
- Support drag-and-drop image file uploads
- Have the text-box be typable as soon as it is generated
- Make default text all caps
- Make text box resize when inputted text is not all visible
- Fix the size of the image to be within a certain range, as right now some images can appear really big or really small
- Generate a mini template library where the user can filter/search based on keywords
- Improve design/UI

Version 2

Version 2 focused on improving the text-box editing experience. The main issues from Version 1 were that text boxes were not immediately ready to type into, moving text boxes was difficult, especially on mobile, and the live webpage text size did not match the downloaded result.

AI Model Used

Tim - I used Claude Opus 4.6 in medium thinking mode.

Version 2 — Iteration 1

Prompt Used

We are working on V1 of a meme generator web app.

Current behavior:

- Users click the image/canvas to add a new text box.
- After adding the text box, the user has to manually click inside the text area before typing.
- This slows down the workflow.

Goal:

When a new text box is created, the text area should immediately become active so the user can start typing right away.

Requirements:

1. After clicking to create a new text box, focus the textarea automatically.
2. Put the cursor inside the text area immediately.
3. If default placeholder text exists, make it easy to replace.
4. This should work on desktop.
5. It should also work as well as possible on mobile/touch devices.
6. Do not break dragging, resizing, deleting, font selection, border toggle, or exporting.
7. Keep the current structure and make a minimal clean change.

Please inspect:

- TextBox.js
- TextBoxManager.js
- any related event handling files

Then update the code so every newly created text box is ready for immediate typing. Explain what files you changed and why.

Changes Made

TextBox.js

- Removed `textarea.focus()` from `_buildDOM()` because it was being called too early.
- Added `TextBox.prototype.focusTextarea()`.
- The new focus method calls `textarea.focus()` twice:
 - once immediately
 - once with `setTimeout(0)`
- This helps support both mobile and desktop focus behavior.

TextBoxManager.js

- Added `tb.focusTextarea()` at the end of `createTextBox()`, after `tb.select()`.
- Added a `touchend` listener on the container so tapping the canvas on mobile can also create a text box and focus it.

Observations

Goals Achieved

- New text boxes are closer to being immediately ready for typing.
- Text focus is handled after the text box is created and selected.
- Mobile tap support was improved by adding a touch event listener.
- Existing features were not intentionally changed.

Goals Not Achieved / Issues to Watch

- Mobile behavior may still vary depending on the browser.
- There is a possible risk that both `touchend` and `mousedown` could fire from one tap, creating duplicate text boxes.
- Further testing is needed on desktop and mobile.

Next Steps

- Test whether one tap creates only one text box.
- Test whether the keyboard opens automatically on mobile.
- Move on to improving text-box movement with a dedicated move handle.

Version 2 — Iteration 2

Prompt Used

We are working on V1 of a meme generator web app.

Current issue:

- Text boxes can currently be moved, but it is hard to move them accurately.
- On mobile, moving text boxes with fingers/thumbs is especially difficult.
- Users may accidentally edit text when they are trying to move it, or struggle to grab the right area.

Goal:

Add a clear, easy way to move text boxes.

Preferred design:

Add a visible "Move" handle/button to each text box toolbar, such as:

- a button labeled "Move"
- or an icon-like button with text such as "↕ Move" or "✦"

Behavior:

1. Users should be able to drag the text box by pressing/clicking/touching the Move handle.
2. The move handle should work well with mouse and touch input.
3. Use pointer events if possible so desktop and mobile can share the same logic.
4. The text area should remain editable when the user taps/clicks inside it.
5. Dragging should not accidentally trigger text editing.

6. Moving should keep the text box inside the image/container bounds.
7. Existing resize handles should still work.
8. Existing delete, font select, border toggle, and export behavior should still work.
9. Keep the current architecture and avoid rewriting the entire app.

Please inspect:

- TextBox.js
- DragResize.js
- TextBoxManager.js
- styles.css

Then implement a clean move handle/button for each text box that makes moving text boxes easier on desktop and mobile. Explain what files you changed and why.

Changes Made

TextBox.js

- Added a ✨ Move button as the first item in the text-box toolbar.
- Exposed the move button as `this.moveBtn` so `DragResize.js` can attach movement logic to it.

DragResize.js

- Removed the old behavior where users could drag from almost anywhere on the text box.
- Added `pointerdown`, `pointermove`, and `pointerup` events to the move button.
- Used `setPointerCapture()` so dragging continues even if the pointer moves away from the button.
- Kept resize handles unchanged.

styles.css

- Changed `.text-box` cursor from `move` to `default`.
- Added `.move-handle` styling.
- Added `touch-action: none` to the move handle so mobile browsers do not treat the gesture as page scrolling.

Observations

Goals Achieved

- Text boxes now have a dedicated move button.
- Moving text boxes is more intentional and less likely to interfere with editing text.
- Mobile movement should be easier because the move button works with pointer events.
- The textarea remains focused on text editing instead of acting as a drag surface.
- Existing resize handles were preserved.

Goals Not Achieved / Issues to Watch

- Resize handles are still mouse-event based, so mobile resizing may still be limited.

- The move button improves movement, but toolbar spacing may need UI polish later.
- Need to test whether mobile scrolling conflicts are fully resolved.

Next Steps

- Test moving text boxes on desktop and mobile.
 - Confirm that tapping inside the textarea edits text instead of dragging.
 - Move on to fixing live font resizing.
-

Version 2 — Iteration 3

Prompt Used

We are working on V1 of a meme generator web app.

Current behavior:

- Users upload an image.
- Users click the image to create editable text boxes.
- Text boxes can be resized.
- When the meme is downloaded, the text scales correctly based on the resized box.
- However, while editing on the website, resizing the text box only changes the box size, not the visible text size.

Goal:

Fix the live editor so that when a user resizes a text box, the text inside visually resizes immediately on the website, matching what will appear in the downloaded PNG.

Important requirements:

1. The live text size should update while resizing, not only after export.
2. The export/download result should still match the live editor as closely as possible.
3. Do not rewrite the whole app.
4. Keep the current file structure and architecture as much as possible.
5. Make the smallest clean change needed.
6. Avoid breaking dragging, deleting, font selection, border toggle, or exporting.

Please inspect the relevant files, especially:

- TextBox.js
- DragResize.js
- TextBoxManager.js
- Exporter.js
- styles.css

Then update the code so text resizing is reflected both live in the editor and in the exported meme. Explain what files you changed and why.

Changes Made

TextBox.js

- Added `TextBox.prototype.syncFontSize()`.
- The method reads the text box height and sets the textarea font size using `Math.max(12, Math.floor(height * 0.4))`.
- Called `this.syncFontSize()` at the end of `_buildDOM()` so new text boxes start with a calculated font size.

DragResize.js

- Added `textBox.syncFontSize()` during the resize mousemove handler.
- The goal was to update the visible text size live while resizing the box.

Observations

Goals Achieved

- A font-size syncing method was added.
- Live editor font size and exporter font size were intended to use the same calculation.
- The implementation was small and did not change many files.

Goals Not Achieved / Issues Found

- The text still appeared to stay the same size during resizing.
- The visual font resizing was not clear enough or did not work as intended.
- The app still needs a more direct text-size system instead of relying only on box height.
- Export and live editor may still not fully match.

Next Steps

- Add actual font-size state to each text box.
- Add visible text-size controls in the toolbar.
- Make the text box resize and font size work together more clearly.
- Make the exporter read the actual live font size instead of using a hidden formula.

Version 2 — Iteration 4

Prompt Used

We are working on V1 of a meme generator web app.

Current state:

- Text boxes can be created on top of the image.
- Text boxes have a toolbar.
- There is now a 🏠 Move button used to move the text box.
- Text boxes can be resized.
- There is a `TextBox.prototype.syncFontSize()` method using `Math.max(12, Math.floor(height * 0.4))`.

- `DragResize.js` calls `textBox.syncFontSize()` during resize.
- However, the visible text in the website editor still looks like the same size when resizing the text box. The previous fix did not solve the issue clearly enough.

New goal:

We need text sizing to be visible and controllable in the editor.

What we want:

1. When the user resizes/drag the resize handles of the text box, the text inside should visibly scale with the text box.
2. Add a clear text-size control in the toolbar, near the ✎ Move button.
3. The text-size control should let the user make the text bigger or smaller manually.
4. When the text size changes manually, the text box should resize/fit around the text better instead of staying awkwardly the same.
5. The live editor should show the current font size clearly.
6. Export/download should use the same font size that is visible in the editor.

Preferred toolbar layout:

- ✎ Move
- Text size controls:
 - A–
 - font size display like "24px"
 - A+
- Existing controls after that:
 - font dropdown
 - border toggle
 - delete

Implementation requirements:

1. Store font size as actual state on each `TextBox` object, for example `this.fontSize`.
2. Do not rely only on CSS font-size or only on `offsetHeight`.
3. When a text box is resized, update `this.fontSize` and `textarea.style.fontSize` so the change is obvious live.
4. When A+ or A– is clicked/tapped, update `this.fontSize`, update `textarea.style.fontSize`, update the visible font-size display, and resize the text box if needed.
5. `Exporter.js` must read the `TextBox` font size state or computed `textarea` font size instead of recalculating a different size secretly.
6. The live website version and downloaded PNG should match as closely as possible.
7. Keep the existing ✎ Move button behavior.
8. Keep mobile usability in mind. The A– and A+ buttons should be easy to tap.
9. Do not rewrite the entire app.
10. Keep changes clean and limited to the relevant files.

Important:

The previous formula-based fix did not visually work. Please inspect why the textarea may not be visually changing size. Possible causes could include CSS overrides, textarea sizing, line-height, transform issues, cached styles, or `Exporter.js` using a different sizing source.

Files to inspect:

- `TextBox.js`
- `DragResize.js`
- `Exporter.js`
- `styles.css`
- `TextBoxManager.js` if needed

After updating the code, explain:

1. Why the previous `syncFontSize` approach was not enough.
2. What new font-size state/control was added.
3. How resizing the text box changes the live text size.
4. How `A-` and `A+` change the font size.
5. How the text box resizes/fits when font size changes.
6. How `Exporter.js` now matches the live editor.
7. What files changed.

Observations

Goals Achieved

- This iteration was designed to replace the weak formula-only approach with a clearer text-size system.
- The prompt required font size to become actual text-box state instead of being hidden inside the exporter.
- The prompt required visible `A-`, font-size display, and `A+` controls.
- The prompt required export to match the live editor by reading the same font-size state or computed style.

Goals Not Achieved / Issues to Verify

- This iteration still needs to be tested after implementation.
- Need to verify that the text visibly changes size on the webpage.
- Need to verify that changing font size also adjusts the text box cleanly.
- Need to verify that exported text matches the live text.
- Need to verify that mobile toolbar buttons are easy to tap.

Next Steps

- Test text resizing with the resize handles.
- Test the `A-` and `A+` controls.
- Compare the webpage version to the downloaded image.
- Test on desktop and mobile.
- Continue improving text-box resizing if the live editor and export still do not match.